Project Documentation: Parameterized Generic Register

Digital Design Lab

Project Overview

This lab implements a parameterized **Generic Register** in Verilog HDL [?]. Registers of this design serve as vital storage building blocks in processor architectures, such as accumulator registers and instruction registers [?]. The module provides synchronous data loading and reset operations driven on the positive edge of a clock signal [?].

Specifications & Functionality

- **Default Bus Width**: `WIDTH = 8` bits (Parameterized) [?].
- **Clock Sensitivity**: Operations trigger synchronously on the rising edge (`posedge clk`) [?].
- **Synchronous Active-High Reset**: When `rst = 1'b1`, `data_out` is cleared to 0 on the rising clock edge [?].
- **Load Control**: When `rst = 1'b0` and `load = 1'b1`, `data_in` is sampled into `data_out` on the rising clock edge [?]. If `load = 1'b0`, the register holds its previous value [?].

Port Summary

Port Name	Direction	Type	Width	Description
`clk`	Input	`wire`	1	System clock signal [?, ?, ?]
`rst`	Input	`wire`	1	Synchronous active-high reset [?, ?, ?]
`load`	Input	`wire`	1	Synchronous load enable control [?, ?, ?]
`data_in`	Input	`wire`	`WIDTH`	Data input vector [?, ?, ?]
`data_out`	Output	`reg`	`WIDTH`	Register output bus [?, ?, ?]

Table 1: Generic Register Port Interface

—

Critical Design Fixes & Analysis

Reset Logic Correction Required

In the initial implementation snippet (`register.v`), the reset condition was written as:

```
if (!rst) data_out <= {WIDTH{1'b0}};
```

This implemented an **active-low** reset [?]. However, according to both the testbench [?] and the specification document [?], the reset signal is defined as **synchronous active-high**.
Changing the condition to `if (rst)` ensures the reset behavior matches the specified requirements [?, ?].

—

Verilog Source Code & Explanations

Corrected Design Module (`register.v`)

Below is the fully corrected implementation for `register.v`:

```
module register #(
    parameter WIDTH = 8
)(
    input  wire             clk,
    input  wire             rst,
    input  wire             load,
    input  wire [WIDTH-1:0] data_in,
    output reg  [WIDTH-1:0] data_out
);

    // Sequential process triggered on clock rising edge
    always @(posedge clk) begin
        if (rst) begin
            data_out <= {WIDTH{1'b0}}; // Synchronous Active-High Reset
        end else if (load) begin
            data_out <= data_in;       // Synchronous Load Data
        end
        // Implicitly retains current value when load == 0
    end

endmodule
```

4.1.1 Design Code Breakdown

- `always @(posedge clk)`: Specifies sequential logic triggered exclusively on the rising clock edge [?, ?].
- `if (rst)`: Evaluates active-high synchronous reset priority [?, ?].
- `data_out <= data_in`: Non-blocking assignment (`<=`) used for sequential register updating [?, ?].

—

Testbench Module (register_test.v)

```
`timescale 1ns / 1ps

module register_test;

  localparam WIDTH = 8;

  reg              clk;
  reg              rst;
  reg              load;
  reg  [WIDTH-1:0] data_in;
  wire [WIDTH-1:0] data_out;

  // Instantiate Unit Under Test (UUT)
  register #(
    .WIDTH (WIDTH)
  ) register_inst (
    .clk      (clk),
    .rst      (rst),
    .load     (load),
    .data_in  (data_in),
    .data_out (data_out)
  );

  // Assertion Check Task
  task expect;
    input [WIDTH-1:0] exp_out;
    begin
      if (data_out !== exp_out) begin
        $display("TEST FAILED");
        $display("At time %0d rst=%b load=%b data_in=%b data_out=%b",
                 $time, rst, load, data_in, data_out);
        $display("data_out should be %b", exp_out);
        $finish;
      end else begin
        $display("At time %0d rst=%b load=%b data_in=%b data_out=%b",
                 $time, rst, load, data_in, data_out);
      end
    end
  endtask

  // Clock Generation (Period = 10ns)
  initial repeat (5) begin
    #5 clk = 1;
    #5 clk = 0;
  end

  % Stimulus Sequence Driving Inputs on Negative Clock Edge
  initial @(negedge clk) begin
    rst=0; load=1; data_in=8'h55; @(negedge clk) expect (8'h55);
    rst=0; load=1; data_in=8'hAA; @(negedge clk) expect (8'hAA);
    rst=0; load=1; data_in=8'hFF; @(negedge clk) expect (8'hFF);
    rst=1; load=1; data_in=8'hFF; @(negedge clk) expect (8'h00);
    $display("TEST PASSED");
    $finish;
  end
```

```
endmodule
```

—

Simulation & Verification

Running Simulation

To execute the simulation using Synopsys VCS [**?**]:

```
vcs register.v register_test.v -R
```

Expected Output Results

Upon applying the active-high reset fix, the simulation outputs the expected test results [**?**]:

```
At time 20 rst=0 load=1 data_in=01010101 data_out=01010101
At time 30 rst=0 load=1 data_in=10101010 data_out=10101010
At time 40 rst=0 load=1 data_in=11111111 data_out=11111111
At time 50 rst=1 load=1 data_in=11111111 data_out=00000000
TEST PASSED
```